

Contents

1	Hello, World!	4
1.1	Write and read a <code>string</code> to/from an <code>iostream</code>	4
1.2	Write and read a <code>map<string, string></code> to/from an <code>iostream</code>	5
1.3	Turtles all the way down	7
2	Primary I/O Classes	8
2.1	<code>Dwm::StreamIO</code>	8
2.2	<code>Dwm::FileIO</code>	8
2.3	<code>Dwm::DescriptorIO</code>	8
2.4	<code>Dwm::BZ2IO</code>	8
2.5	<code>Dwm::GZIO</code>	9
2.6	<code>Dwm::ASIO</code>	9
3	Directly supported data types	10
3.1	integral types	10
3.2	floating point types	10
3.3	enumerated types	10
3.4	<code>std::string</code>	10
3.5	<code>std::vector<bool></code>	10
4	Supported <code>std</code> type templates	11
4.1	<code>std::array<T,N></code>	11
4.2	<code>std::deque<T></code>	11
4.3	<code>std::list<T></code>	11
4.4	<code>std::map<KeyType, MappedType></code>	11
4.5	<code>std::multimap<KeyType, MappedType></code>	11
4.6	<code>std::optional<T></code>	11

4.7	<code>std::pair<FirstType, SecondType></code>	11
4.8	<code>std::set<T></code>	11
4.9	<code>std::multiset<T></code>	11
4.10	<code>std::unique_ptr<T></code>	12
4.11	<code>std::unordered_map<KeyType, MappedType></code>	12
4.12	<code>std::unordered_multimap<KeyType, MappedType></code>	12
4.13	<code>std::unordered_set<T></code>	12
4.14	<code>std::unordered_multiset<T></code>	12
4.15	<code>std::vector<T></code>	12
5	Concepts	13
5.1	Streamable	13
5.2	C++ iostreams	13
5.2.1	<code>HasStreamRead<T></code>	13
5.2.2	<code>HasStreamWrite<T></code>	13
5.2.3	<code>Dwm::IsStreamReadable<T></code>	13
5.2.4	<code>Dwm::IsStreamWritable<T></code>	13
5.3	FILE pointer (cstdio)	13
5.3.1	<code>HasFileRead<T></code>	13
5.3.2	<code>HasFileWrite<T></code>	13
5.4	UNIX descriptors	14
5.4.1	<code>HasDescriptorRead<T></code>	14
5.4.2	<code>HasDescriptorWrite<T></code>	14
5.5	Boost asio sockets	14
5.5.1	<code>HasAsioRead<T></code>	14
5.5.2	<code>HasAsioWrite<T></code>	14
5.6	BZFILE pointer (bzip2)	14
5.6.1	<code>HasBZRead<T></code>	14
5.6.2	<code>HasBZWrite<T></code>	14
5.7	<code>gzFile</code> (libz)	15

CONTENTS	3
5.7.1 HasGZRead<T>	15
5.7.2 HasGZWrite<T>	15
5.8 HasSkipAnnotation<std::meta::info info>	15
5.9 Streamed length	15
5.9.1 Constant streamed length	15
5.9.2 Constant streamed length via static constexpr uint64_t StreamedLength() member	15
5.9.3 Constant streamed length from C++26 reflection	15
6 On-the-wire formats	16
6.1 On-the-wire formats, network byte order (MSB first)	16
6.1.1 16 bit integral types (int16_t, uint16_t)	16
6.1.2 32 bit integral types (int32_t, uint32_t)	16
6.1.3 64 bit integral types (int64_t, uint64_t)	16
6.1.4 float	16
6.1.5 double	16
6.1.6 std::string	17
6.1.7 std::pair<T1, T2>	17
6.1.8 std::vector<T>	17
6.1.9 std::deque<T>	17
6.1.10 std::list<T>	17
6.1.11 std::map<Key, Value>	18
6.1.12 std::multimap<Key, Value>	18
6.1.13 std::unordered_map<Key, Value>	18
6.1.14 std::unordered_multimap<Key, Value>	18
6.1.15 std::set<T>	18
6.1.16 std::multiset<T>	19
6.1.17 std::unordered_set<T>	19
6.1.18 std::unordered_multiset<T>	19

Chapter 1

Hello, World!

1.1 Write and read a string to/from an iostream

The program below writes a `std::string` to a `std::stringstream`, then reads a `std::string` from the `std::stringstream`. The program returns 0 if the string we read matches the string we wrote.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using std::string, std::stringstream;
7     bool success{false};
8     string writeString("Hello, world!"), readString;
9     stringstream ss;
10    if (Dwm::StreamIO::Write(ss, writeString)) {
11        if (Dwm::StreamIO::Read(ss, readString)) {
12            success = (readString == writeString);
13        }
14    }
15    return success ? 0 : 1;
16 }
```

1.2 Write and read a `map<string, string>` to/from an `iostream`

The program below writes a `std::map<std::string, std::string>` to a `std::stringstream`, then reads a `std::map<std::string, std::string>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers holding supported types (`std::string` is an explicitly supported type).

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using stringmap = std::map<std::string, std::string>;
7     stringmap m1 {
8         { "apple",      "fruit"      },
9         { "cat",        "feline"     },
10        { "broccoli",   "vegetable"   },
11        { "dog",        "canine"      },
12        { "kiwi",       "fruit"       },
13        { "cow",        "bovine"      }
14    };
15    bool success{false};
16    std::stringstream ss;
17    if (Dwm::StreamIO::Write(ss, m1)) {
18        stringmap m2;
19        if (Dwm::StreamIO::Read(ss, m2)) {
20            success = (m2 == m1);
21        }
22    }
23    return success ? 0 : 1;
24 }
```

```
1 #include <fstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using stringmap = std::map<std::string, std::string>;
7     std::string tmpFile("./stringmaptest.tmp");
8     stringmap m1 {
9         { "apple", "fruit" },
10        { "cat", "feline" },
11        { "broccoli", "vegetable" },
12        { "dog", "canine" },
13        { "kiwi", "fruit" },
14        { "cow", "bovine" }
15    };
16    bool success{false};
17    std::ofstream os(tmpFile);
18    if (os) {
19        Dwm::StreamIO::Write(os, m1);
20        os.close();
21        std::ifstream is(tmpFile);
22        if (is) {
23            stringmap m2;
24            Dwm::StreamIO::Read(is, m2);
25            success = (m2 == m1);
26            is.close();
27        }
28        std::remove(tmpFile.c_str());
29    }
30    return success ? 0 : 1;
31 }
```

1.3 Turtles all the way down

The program below writes a `std::map<std::string, std::vector<std::string>>` to a `std::stringstream`, then reads a `std::map<std::string, std::vector<std::string>>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers and supported types being “turtles all the way down”: our support is recursive to any depth.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using container = std::map<std::string, std::vector<std::string>>;
7     container m1 {
8         { "fruit",      { "apple", "banana", "kiwi", "orange" } },
9         { "canine",     { "husky", "labrador", "poodle", "wolf", "collie" } },
10        { "feline",     { "cougar", "lion", "panther" } },
11        { "vegetable", { "broccoli", "carrot", "squash" } }
12    };
13
14    bool success{false};
15    std::stringstream ss;
16    if (Dwm::StreamIO::Write(ss, m1)) {
17        container m2;
18        if (Dwm::StreamIO::Read(ss, m2)) {
19            success = (m2 == m1);
20        }
21    }
22    return success ? 0 : 1;
23 }
```

Chapter 2

Primary I/O Classes

2.1 Dwm::StreamIO

source type: `std::istream`
sink type: `std::ostream`

Uses the [HasStreamRead](#) and [HasStreamWrite](#) concepts to detect the presence of stream I/O support within a user defined type. In the absence of C++26 reflection, this support requires a user defined type to contain these two member functions:

```
std::istream & Read(std::istream & is)
std::ostream & Write(std::ostream & os) const
```

2.2 Dwm::FileIO

source type: `FILE *`
sink type: `FILE *`

Uses the [HasFileRead](#) and [HasFileWrite](#) concepts to detect the presence of C stdio support (reading and writing from/to `FILE *`) in a user defined type.

```
size_t Read(FILE *f)
size_t Write(FILE *f) const
```

2.3 Dwm::DescriptorIO

source type: `int`
sink type: `int`

Uses the [HasDescriptorRead](#) and [HasDescriptorWrite](#) concepts to detect the presence of descriptor I/O support (reading and writing from/to UNIX file descriptors) inside a user defined type.

```
ssize_t Read(int fd)
ssize_t Write(int fd) const
```

2.4 Dwm::BZ2IO

source type: `BZFILE *`
sink type: `BZFILE *`

Uses the [HasBZRead](#) and [HasBZWrite](#) concepts to detect the presence of bzip2 I/O support (reading and writing from/to `BZFILE *`) inside a user defined type.

```
int BZRead(BZFILE *)
int BZWrite(BZFILE *) const
```

2.5 Dwm::GZIO

gzFile source type: gzFile
 sink type: gzFile

```
int Read(gzFile)
int BZWrite(gzFile) const
```

2.6 Dwm::ASIO

Boost asio sockets

Chapter 3

Directly supported data types

Directly supported data types are those types for which we have static member functions to read the type from a source and write the type to a sink. These types include integral types, floating point types, enumerated types, `std::string` and `std::vector<bool>`.

3.1 integral types

The category of types for which `std::is_integral_v<T>` evaluates to `true`.

For multi-byte types in this category, we use an MSB-first representation on-the-wire.

3.2 floating point types

The category of types for which `std::is_floating_point_v<T>` evaluates to `true`.

We use XDR for the on-the-wire representation of `float` and `double`.

3.3 enumerated types

The category of types for which `std::is_enum_v<T>` evaluates to `true`.

3.4 `std::string`

3.5 `std::vector<bool>`

Chapter 4

Supported `std` type templates

4.1 `std::array<T, N>`

We can stream any instances of `std::array<T, N>` where `T` is streamable.

4.2 `std::deque<T>`

We can stream any instances of `std::deque<T>` where `T` is streamable.

4.3 `std::list<T>`

We can stream any instances of `std::list<T>` where `T` is streamable.

4.4 `std::map<KeyType, MappedType>`

We can stream any instances of `std::map<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.5 `std::multimap<KeyType, MappedType>`

We can stream any instances of `std::multimap<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.6 `std::optional<T>`

We can stream any instances of `std::optional<T>` where `T` is streamable.

4.7 `std::pair<FirstType, SecondType>`

4.8 `std::set<T>`

We can stream any instances of `std::set<T>` where `T` is streamable.

4.9 `std::multiset<T>`

We can stream any instances of `std::multiset<T>` where `T` is streamable.

4.10 `std::unique_ptr<T>`

We can stream instances of `std::unique_ptr<T>` where `T` is streamable and the `unique_ptr`'s `deleter_type` is `std::default_delete<T>`. We use the `deleter_type` test to infer that the `std::unique_ptr<T>` points to a single object and not to an array. This can of course be incorrect, and hence should not be relied on as a general mechanism.

4.11 `std::unordered_map<KeyType, MappedType>`

We can stream any instances of `std::unordered_map<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.12 `std::unordered_multimap<KeyType, MappedType>`

We can stream any instances of `std::unordered_multimap<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.13 `std::unordered_set<T>`

We can stream any instances of `std::unordered_set<T>` where `T` is streamable.

4.14 `std::unordered_multiset<T>`

We can stream any instances of `std::unordered_multiset<T>` where `T` is streamable.

4.15 `std::vector<T>`

We can stream any instances of `std::vector<T>` where `T` is streamable.

Chapter 5

Concepts

5.1 Streamable

5.2 C++ iostreams

5.2.1 HasStreamRead<T>

Evaluates to true if `T` has a member function with signature:

```
std::istream & Read(std::istream & is)
```

Required semantics

- Must populate `*this` with contents read from `is`
- Must return `is`

5.2.2 HasStreamWrite<T>

Evaluates to true if `T` has a member function with signature:

```
std::ostream & Write(std::ostream & os) const
```

Required semantics

- Must write the contents of `*this` to `os`
- Must return `os`

5.2.3 Dwm::IsStreamReadable<T>

5.2.4 Dwm::IsStreamWritable<T>

5.3 FILE pointer (cstdio)

5.3.1 HasFileRead<T>

Evaluates to true if `T` has a member function with signature:

```
size_t Read(FILE *f)
```

Required semantics

- Must populate `*this` with contents read from `f`
- Must return a non-zero `size_t` on success, 0 on failure

5.3.2 HasFileWrite<T>

Evaluates to true if `T` has a member function with signature:

```
size_t Write(FILE *f) const
```

Required semantics

- Must write the contents of `*this` to `f`
- Must return a non-zero `size_t` on success, 0 on failure

5.4 UNIX descriptors

5.4.1 HasDescriptorRead<T>

Evaluates to true if `T` has a member function with signature:

```
ssize_t Read(int fd)
```

Required semantics

- Must populate `*this` with contents read from `fd`
- Must return a positive `ssize_t` on success, `-1` on failure

5.4.2 HasDescriptorWrite<T>

Evaluates to true if `T` has a member function with signature:

```
ssize_t Write(int fd) const
```

Required semantics

- Must write the contents of `*this` to `fd`
- Must return a positive `ssize_t` on success, `-1` on failure

5.5 Boost asio sockets

5.5.1 HasAsioRead<T>

5.5.2 HasAsioWrite<T>

5.6 BZFILE pointer (bzip2)

5.6.1 HasBZRead<T>

Evaluates to true if `T` has a member function with signature:

```
int BZRead(BZFILE *bzf)
```

Required semantics

- Must populate `*this` with contents read from `bzf`
- Must return a positive `int` on success, `-1` on failure

5.6.2 HasBZWrite<T>

Evaluates to true if `T` has a member function with signature:

```
int Write(BZFILE *bzf) const
```

Required semantics

- Must write the contents of `*this` to `bzf`
- Must return a positive `int` on success, `-1` on failure

5.7 `gzFile (libz)`

5.7.1 `HasGZRead<T>`

Evaluates to true if `T` has a member function with signature:

```
int Read(gzFile gzf)
```

Required semantics

- Must populate `*this` with contents read from `gzf`
- Should return the number of decompressed bytes read on success, -1 on failure

5.7.2 `HasGZWrite<T>`

Evaluates to true if `T` has a member function with signature:

```
int Write(gzFile gzf) const
```

Required semantics

- Must write the contents of `*this` to `gzf`
- Should return the number of uncompressed bytes written on success, -1 on failure

5.8 `HasSkipAnnotation<std::meta::info info>`

Evaluates to true if the given reflection `info` has an annotation of the form `[ [=Dwm::skip_io]]`

This requires C++26 reflection, specifically P2996 and P3394.

5.9 Streamed length

5.9.1 Constant streamed length

5.9.2 Constant streamed length via `static constexpr uint64_t StreamedLength()` member

5.9.3 Constant streamed length from C++26 reflection

When using C++26 reflection for serialization and deserialization of instances of a given type, we can determine at compile time whether or not the streamed length of the type is constant, and hence whether or not we can leverage `StreamIO` to provide `DescriptorIO`, `FileIO`, `BZ2IO`, `GZIO` and `ASIO` functionality automatically (via a pass through a `std::stringstream`).

Chapter 6

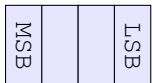
On-the-wire formats

6.1 On-the-wire formats, network byte order (MSB first)

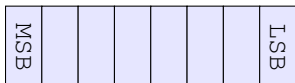
6.1.1 16 bit integral types (`int16_t`, `uint16_t`)



6.1.2 32 bit integral types (`int32_t`, `uint32_t`)

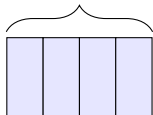


6.1.3 64 bit integral types (`int64_t`, `uint64_t`)



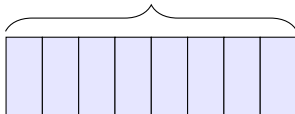
6.1.4 `float`

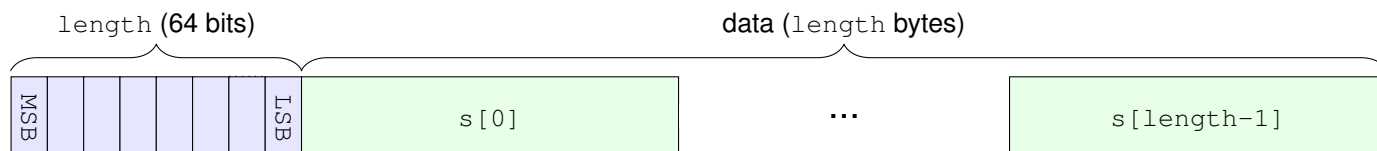
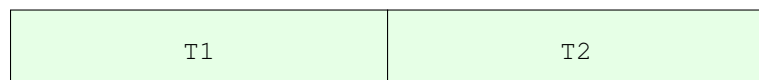
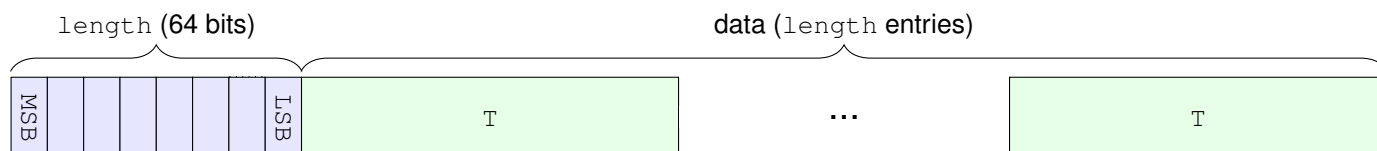
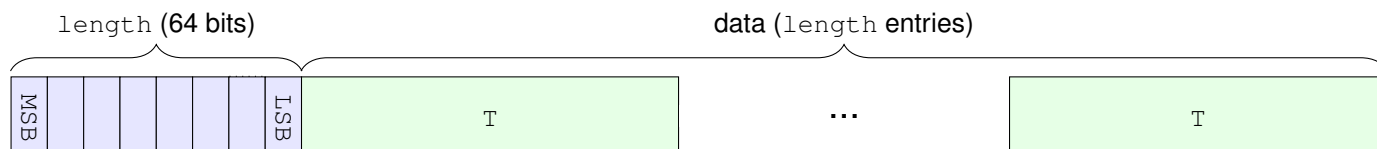
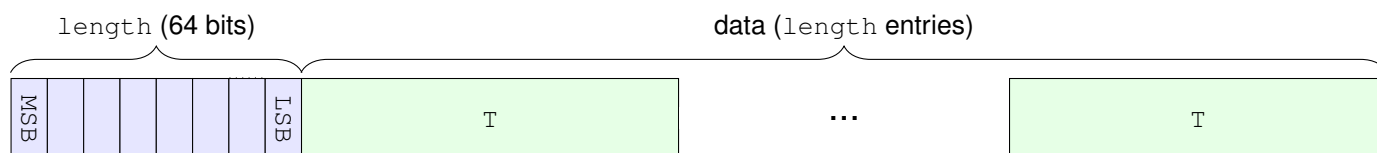
4 bytes, RFC4506

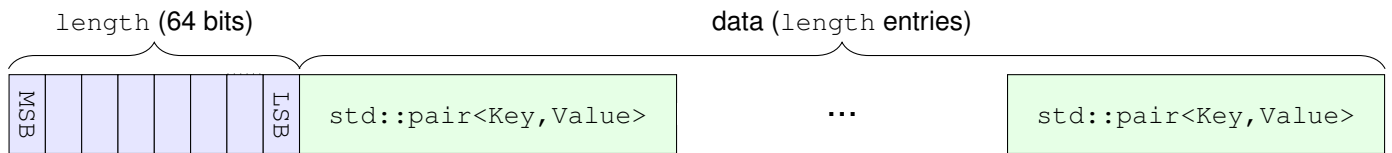
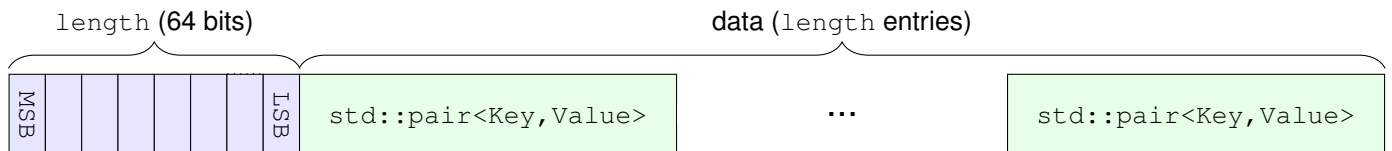
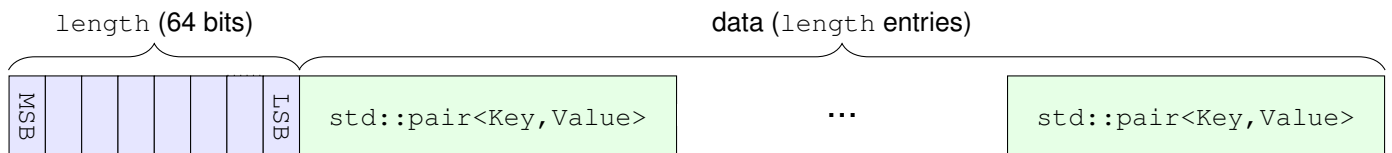
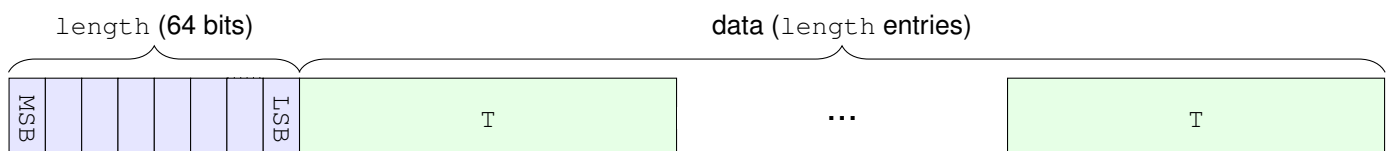


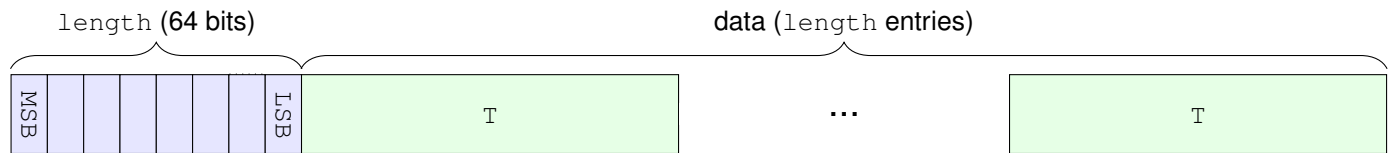
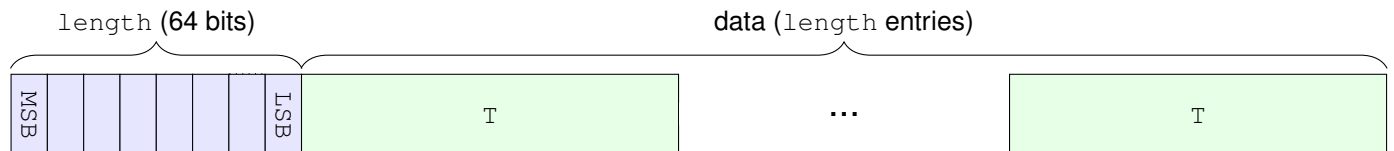
6.1.5 `double`

8 bytes, RFC4506



6.1.6 `std::string`**6.1.7 `std::pair<T1, T2>`****6.1.8 `std::vector<T>`****6.1.9 `std::deque<T>`****6.1.10 `std::list<T>`**

6.1.11 `std::map<Key, Value>`**6.1.12 `std::multimap<Key, Value>`****6.1.13 `std::unordered_map<Key, Value>`****6.1.14 `std::unordered_multimap<Key, Value>`****6.1.15 `std::set<T>`**

6.1.16 `std::multiset<T>`**6.1.17 `std::unordered_set<T>`****6.1.18 `std::unordered_multiset<T>`**